

# Iteration 2 — Presentation Script

---

Team Left4Dead — Project D  
Course Support & Activity Dashboard

Matched to the Iteration 2 Report  
US-4, US-5, US-6 in progress · US-16 deferred

Kantanut Utamapongchai · Thanawin Thanapornthawan  
Phutarak Wongpitak · Pakhin Daonan

# Fix the report before you present

This script now matches your Iteration 2 report. But five things in that report will contradict what you say out loud. Fix them first, or the script will not save you.

## 1 — THE REPOSITORY LINK IS WRONG

Section 1 says `https://www.poolsuite.net/app`. That is a music website, not your project. The report says the repository is **mandatory** for Iteration 2. It should be `https://github.com/ThanawinTH/ISP-Left4Dead`.

## 2 — BOTH CRITICAL PATHS ARE FROM THE RESTAURANT EXAMPLE

Section 1.1 says "Menu Create/Edit implementation (US-2) → Availability update logic (US-3) → Booking module". Section 1.2 says "Finalize menu-item data model → Implement Create/Edit Menu Item". That is Milk's Restaurant, the lecturer's own teaching example. It is not your project.

**Your Iteration 2 critical path is:** Finalise role and assignment database schema → Implement role checking (SRS-7) → Implement Create Assignment (US-5) → Assignment list and visibility → Testing → Report.

## 3 — US-5 AND US-6 STILL SAY "MENU ITEM"

Your story descriptions read "*As an administrator, I want to create a menu item*" and "*update menu item availability*". The Notes column next to them correctly talks about assignments. So the table disagrees with itself. Rewrite both to match your real stories — and the actor is the **lecturer**, not an administrator, which is what your own scope log says.

## 4 — THE RETROSPECTIVE IS ABOUT THE WRONG PROJECT

It mentions "US-3 validation logic (rejecting invalid availability changes)" and "the customer-facing menu view". Your US-3 is *join a classroom by code*. There is no menu view in your system. Rewrite all three rows using what really happened this iteration.

## 5 — "TESTING: NONE APPLICABLE" IS THE MOST EXPENSIVE LINE IN THE DOCUMENT

Criterion E is worth **20%** at Iteration 2. The rubric's fail band reads "*No meaningful testing; evaluation absent or wholly unsupported.*" Writing "None Applicable" puts you there by your own words.

It is also not true. You tested in Iteration 1 — Kantanut found the double-click bug by typing the join code like a real user. Even an hour of writing down what you checked this iteration moves this criterion up a band. **Do this before you submit.**

## SMALLER THINGS

- Kantanut's surname is spelled **Kantanat** on the cover and **Kantanut** in the table.
- Sections 6 and 7 — Risk Register and Individual Contribution Log — are in your table of contents but had no content. Criterion G checks that the contribution log matches the commit history, so this one matters.
- Your assignment fields are now title, description, due date and assigned TA. SRS-9 says title, type, due date and visibility. One of the two needs to change, or they contradict each other.

## What changed in this script

| Change                                                          | Why                                                                                                 |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| "Activity" is now "assignment" everywhere                       | Your scope log renamed it                                                                           |
| Owners corrected                                                | US-5 is Phutarak + Thanawin. US-6 is Phutarak + Kantanut. Not what I had before                     |
| Everything is presented as <b>work in progress</b>              | Your report says In Progress, not Done. The guideline expects WIP and asks you to label it honestly |
| US-16 is <b>deferred</b>                                        | Your report says the team has not agreed to include it                                              |
| Assignment fields are title, description, due date, assigned TA | From your scope log                                                                                 |
| Testing segment rewritten                                       | To be honest about where you are, with a better version if you write the notes first                |

### HOW TO SAY IT

Sentences are short on purpose. **Stop at every full stop.** A short pause sounds calm, and gives you time to breathe.

Practise these words out loud five times before the day: **authentication · authorisation · architecture · middleware · traceability · acceptance criteria**

## Who speaks, and for how long

| # | Segment                   | Time        | Speaker                                                     | Marks | Why this person                                             |
|---|---------------------------|-------------|-------------------------------------------------------------|-------|-------------------------------------------------------------|
| 1 | Recap and goal            | 1:00        | Kantanut                                                    | B, G  | The opening rotates. Phutarak opened at the midterm         |
| 2 | Requirements and scope    | 1:00        | Pakhin                                                      | B     | He owns the role model, which is what the scope log changed |
| 3 | Live demo                 | 2:30        | Thanawin                                                    | D, B  | He built the member page and the create form                |
| 4 | Design decision           | 1:00        | Phutarak                                                    | C     | He built the assignments table and both endpoints           |
| 5 | Individual parts × 4      | 2:00        | Everyone, 30s each                                          | A     | Everyone must speak for their own work                      |
| 6 | Testing                   | 1:00        | Kantanut                                                    | E     | He is the QA member                                         |
| 7 | Reflection and next steps | 0:30        | Pakhin                                                      | E, A  | Not the same person who opened                              |
|   | <b>Total</b>              | <b>9:00</b> | Thanawin 3:00 · Kantanut 2:30 · Pakhin 2:00 · Phutarak 1:30 |       |                                                             |

## Segment 1 · Recap and goal

KANTANUT UTAMAPONGCHAI

1:00 | Slide 01

### LOOK AT THE ROOM, NOT THE SCREEN.

Good morning. We are Team Left4Dead. Our project is Challenge D, the Course Support and Activity Dashboard.

The problem in one line. Course staff keep deadlines in many places. Spreadsheets, calendars, chat, email. So nobody can see what is late, or who owns it. Our system puts it in one place.

### POINT AT SLIDE 01.

In Iteration 1 we finished three user stories. Create a classroom. Make a join code. Join with that code.

For Iteration 2 our capacity was four user stories. We planned US-4, US-5 and US-6. All three are **in progress**, and we will show you how far each one is.

We also had US-16, Google sign-in. We **deferred** it. The team had not agreed on the schedule for it, and we chose to finish the role work properly instead of half-finishing two things.

### SLOW DOWN. THIS IS THE POINT OF THE WHOLE PRESENTATION.

Our goal this time was clear. **We wanted roles to mean something.**

In Iteration 1, everyone who joined became a Student. The system treated everyone the same. Now a lecturer can promote someone to TA or university staff. The system checks that role. And staff can create assignments. This supports Sub-Goals SG-1 and SG-2 in our SRS.

Thanawin will show you what works. First, Pakhin on the requirements.

≈ 210 words · 1:00

### SAYING "IN PROGRESS" EARLY IS THE RIGHT MOVE

The guideline says work in progress is expected. What it forbids is showing an unfinished feature as finished. Saying it in the first minute means nobody can catch you with it later.

## Segment 2 · Requirements and scope

PAKHIN DAONAN

1:00 | Slide 02

### SAY THE IDS CLEARLY. DO NOT READ THE SLIDE.

Three user stories drove this iteration.

**US-4.** A lecturer can promote a member to TA or university staff. It covers SRS-6 and SRS-7. SRS-6 says only the classroom owner can change a role. Our acceptance criteria: the owner can change a role, anyone else is

refused, and the owner cannot change their own row. A classroom must always keep someone who can give out roles.

**US-5.** A lecturer can create an assignment. It covers SRS-8, SRS-9 and SRS-10. Only a lecturer or staff can create one. Required fields are title, description, due date, and the TA who will check it. If a field is missing, the system shows a message and saves nothing.

**US-6.** Staff can update an assignment's status. It covers SRS-8 and SRS-11.

**SCOPE CHANGES. SAY ALL FOUR, BRIEFLY. THIS SECTION IS GRADED.**

We made four changes to scope, and all four were clarifications. We did not add any new user story.

First, we renamed "activity" to "assignment". "Activity" was too broad, and "assignment" matches what we designed in Figma.

Second, US-5 and US-6 now focus on the lecturer. The lecturer decides what the assignment is. TAs check the students' work. They do not decide the assignment.

Third, an assignment now carries a title, a description, a due date, and the TA responsible for checking it. That last field came from the Figma design.

Fourth, we split TA and university staff into two separate roles. They have different jobs in a course, so they should have different permissions.

Nothing was dropped, because nothing was added. Our out-of-scope list has not changed.

*≈ 275 words · 1:00, keep moving*

**SAY "WE DID NOT ADD ANY NEW USER STORY"**

The scope log exists to catch scope creep. Four entries could look like creep. Saying plainly that all four were clarifications, with nothing added, turns the same table into evidence of control.

## Segment 3 · Live demo

### READ THIS FIRST

Do not show three features one by one. Show **one chain**, where changing a role changes what a person can do. That chain is SRS-7.

All three stories are in progress. So at each step, say what works and what does not. Three accounts, signed in before you start.

### THANAWIN THANAPORNTHAWAN

2:30 | Live app

#### LECTURER WINDOW. ALREADY INSIDE A CLASSROOM.

I am signed in as the lecturer who owns this classroom. This is the Members page. It is new this iteration.

#### POINT AT THE TABLE.

Here is everyone in the class, with their role. Each row has an Edit Role link. I can see it because I own this classroom. And it never appears on my own row. That is SRS-6.

#### SWITCH TO THE STUDENT WINDOW BEFORE PROMOTING ANYONE.

This is a second account. A Student in the same class. Same page, same list. But no Edit Role anywhere.

The page hides it. The server would refuse the change anyway. So the page only hides something that would fail.

#### BACK TO THE LECTURER WINDOW. PROMOTE THAT STUDENT TO TA.

Now I promote this person to TA. The badge changes. It saves straight away.

#### STUDENT WINDOW. REFRESH. PAUSE — LET THEM SEE IT.

Same account. No new login. And now the system treats this person differently.

Nothing changed about them. Only one row in the database changed. But the system's answer changed with it. That is what we mean by roles meaning something.

#### NOW THE ASSIGNMENT PART. BE HONEST ABOUT HOW FAR IT GOES.

Next, assignments. **FILL: show what works today — for example the Create Assignment form, the validation message when a field is missing, and the assignment appearing after it saves**.

The required fields are title, description, due date, and the TA who will check it. SRS-9 requires them. If one is missing, SRS-10 shows a message and saves nothing.

#### THE HONEST LINE. SAY IT CLEARLY. DO NOT RUSH IT.

US-5 and US-6 are **work in progress**. What works today is **FILL: list it**. What is not finished is **FILL: list it**. We plan to finish it by **FILL: when**.

Phutarak will explain the design decision behind this.

≈ 290 words plus clicking · practise to 2:30

### DEMO RULES

Three accounts signed in before you start: the owner, a student you will promote, and a student who stays a student. The app already running. A backup video on the desktop showing the same thing. If something freezes for five seconds, switch to the video. A technical problem does not buy extra time.

## Segment 4 · Design decision

PHUTARAK WONGPITAK

1:00 | Slide 04

### SLIDE 04. ONE PICTURE, ONE DECISION.

Our architecture has not changed. It is a modular monolith, using MVC. One Express app and one MySQL database, both in Docker Compose. Inside, it is split into Model, View and Controller.

I built the assignments table and the two endpoints. One to create an assignment, one to update its status.

Our design decision this iteration was this. **Where do we stop a student from creating an assignment?**

We could hide the button. That is easy, and we do hide it. But that is only what the user sees. Anyone can call our API directly, without our page. So the real check has to be on the server, before the code that writes to the database.

### NOW THE COST. THIS IS THE PART THEY ARE MARKING.

The cost is that the rule now lives in two places. The page hides the button, and the server refuses the request. That is duplication, and if we change the rule we have to change both.

We accepted that, because the two jobs are different. The page is there so a student is not offered something that will only fail. The server is there so the data is safe. One is kindness. The other is security.

The database supports this. The memberships table keeps one row per person, per classroom. So the server always has one clear answer when it asks what your role is.

Our limitation: **FILL — for example, the assignment table does not link to submissions yet, so we cannot check who has handed work in**.

≈ 265 words · 1:00

### WHY THIS DECISION IS A GOOD ONE TO PRESENT

It is true, it is yours, and it has a real cost you can name. "Kindness versus security" is a sentence the assessor will remember, and it is the same idea as the client-side double-click fix you reported at the midterm.

## Segment 5 · Each person

Thirty seconds each. Say five things: **what you owned**, **the course idea you used**, **one decision**, **one thing you learned**, **your next task**. Stand up for your turn. This is where Criterion A is won.

### 1. PAKHIN DAONAN

0:30 | US-4 back end

I owned the role model and the back end for US-4. That is the role column in memberships, the promote endpoint, and the permission checks. My evidence is `schema.sql`, `members.js`, and commits **FILL**.

The course idea I used is putting rules in the database, not only in the code. My decision: the API never lets anyone assign the lecturer role. You can only assign student, TA or staff. A classroom must keep the person who created it. It is safer to make a bad state impossible than to check for it every time.

What I learned: changing the database is also changing the requirements. When I split TA and staff into two roles, the SRS and the traceability matrix were both wrong until I updated them. Next iteration I own **FILL**.

≈ 150 words

### 2. PHUTARAK WONGPITAK

0:30 | US-5 and US-6 back end

I owned the back end for US-5 and US-6. That is the assignments table, the endpoint that creates an assignment, and the endpoint that updates its status. My evidence is **FILL: filenames** and commits **FILL**.

The course idea is thinking about security in the design, not after the code. My decision was to check permission on the server, not only in the page. I explained the reason a moment ago.

What I learned is the difference between two words. **Authentication** means the system knows who you are. **Authorisation** means it knows what you may do. At the midterm we said we checked roles on every request. In the code we had the first one, and not the second. Building these endpoints made that difference real to me. Next iteration I own **FILL**.

≈ 155 words

### 3. THANAWIN THANAPORNTHAWAN

0:30 | Front end + documents

I owned the front end for US-4 and US-5. That is the member page with the role dropdown, and the Create Assignment form. I also updated the SRS and the traceability matrix. My evidence is `members.html`, `members.js`, and commits **FILL**.

The course idea is tracing a requirement to the screen. I hide Edit Role from people who are not the owner because SRS-6 says so. Not because it looks cleaner.

My decision was to hide the link, not grey it out. A greyed-out button tells people about something they will never be able to use.

What I learned: hiding a button is not security. The server still has to refuse, because anyone can call the API directly. Next iteration I own **FILL**.

≈ 145 words

#### 4. KANTANUT UTAMAPONGCHAI

0:30 | US-6 front end + QA

I owned the status control on the front end for US-6, and the testing for this iteration. My evidence is **FILL:** **filenames** and commits **FILL**.

The course idea is testing against the acceptance criteria, including the cases that should fail. My decision was to test the paths that should be refused first, not last — a non-owner changing a role, and a student creating an assignment.

What I learned: **FILL — one real thing. For example, a case that only failed when I called the API directly instead of clicking the page**. Next iteration I own **FILL**.

≈ 135 words

## Segment 6 · Testing

### READ THIS BEFORE YOU WRITE THE SEGMENT

Your report says "**None Applicable**" for testing. Criterion E is 20% of this iteration, and the rubric's fail band is *"No meaningful testing; evaluation absent or wholly unsupported."*

Below are two versions. **Version A** is what to say if you write down your test notes before the presentation — one hour of work. **Version B** is what to say if you do not. Version A is worth a whole band more. Please do Version A.

### KANTANUT — VERSION A

1:00 | if you write the test notes

#### SHOW THE DOCUMENT ON SCREEN. THE DOCUMENT IS THE POINT.

This iteration our testing is written down. At the midterm it was not.

We have a test note in the `docs` folder. Each row has the case, the requirement ID, the steps, what we expected, what happened, and pass or fail.

**FILL: how many cases, how many passed, how many problems you found** .

We test the path that should work, and the paths that should fail. A non-owner trying to change a role. The owner's own row. A missing due date. A student trying to create an assignment.

The main problem we found was **FILL: what broke, how you found it, how you fixed it** .

#### SLOW DOWN. THIS PARAGRAPH EARNS THE MARK.

We are honest about two limits. First, our testing is manual. A person runs a checklist by hand. It does not grow well, and it cannot run on every commit. Second, US-5 and US-6 are still in progress, so some cases are written but not run yet.

We also closed a limit from the midterm. Our double-click fix only worked in the browser. It helped the user, but it did not protect the data. **FILL: say if the server fix is done, or still open** .

≈ 235 words · 1:00

### KANTANUT — VERSION B

1:00 | if you do not

#### DO NOT SAY "NONE APPLICABLE" OUT LOUD. SAY WHAT YOU ACTUALLY DID, AND WHAT IS MISSING.

I have to be honest about testing this iteration. We do not have a written test document yet, and that is a gap we know about.

What we did do was check each story by hand as we built it. For US-4 I checked that a non-owner is refused, and that the owner cannot change their own row. **FILL: add the other real checks you ran** .

What is missing is the record. We ran the checks, but we did not write down the case, the requirement, and the result. So we cannot show you evidence, only tell you.

We know why that matters. A test nobody wrote down cannot be repeated, and it cannot be reviewed. It is also the same mistake we made in Iteration 1, and we said then that we would fix it.

So our first task in the next iteration is the test document, covering Iteration 1 and 2 together, written from the acceptance criteria in each user story.

*≈ 185 words · 1:00*

#### **WHY VERSION B IS STILL WORTH SAYING**

It will not score as well as Version A. But naming your own gap, explaining why it matters, and giving a dated plan is the difference between the fail band and the D band. Saying "none applicable" and moving on is the worst of the three options.

## Segment 7 · Reflection and next steps

PAKHIN DAONAN

0:30

Three short things.

**What went well.** We split the work clearly between back end and front end, and everyone knew what they owned. The order also worked. We did the database first, so nothing was blocked waiting for it.

**What did not.** **FILL — the honest one, about this iteration**. And we did not agree early enough about US-16, so its schedule stayed unclear for most of the sprint.

**Next.** Decide US-16 in the first meeting, not the last. **FILL: one or two more**. And one habit we are keeping. Every task now has commits from the person who did it. So the repository shows our work, not just our word.

Thank you. We are happy to take questions.

≈ 150 words · 0:30

### REWRITE THE RETROSPECTIVE IN YOUR REPORT TO MATCH THIS

Your current table talks about "US-3 validation logic" and "the customer-facing menu view". Neither exists in your project. Whatever Pakhin says here must be what the report says too — Criterion G checks that they agree.

## Questions

The person who did the work answers. If you do not know, say so, and say who does. Answer in one breath, then stop.

**Your report says all three stories are in progress. So what is actually finished?** Thanawin

"**FILL: be specific. Name what a user can do end to end today, and what they cannot.** US-4 is the furthest along — you saw the full path in the demo. US-5 and US-6 have the database and the endpoints, and the front end is **FILL: how far**. We chose to report them as in progress rather than call them done."

**You changed scope four times. Is that not scope creep?** Pakhin

"We did not add a single user story. All four entries are clarifications of stories we already had. Renaming activity to assignment. Saying the lecturer decides the assignment, not the TA. Adding the fields we found in our own Figma design. And splitting TA and staff into two roles. Scope creep is when new features are approved without checking the cost. Nothing new was approved, so nothing had to be dropped."

**Why did you defer US-16?** Phutarak

"Two reasons. It was the riskiest item in our plan — nobody on the team had used Google sign-in before, and it sits in the part that everything else depends on. And we had not agreed as a team on when to do it. We decided to finish the role work properly instead of half-finishing two things. Passwords are still hashed with a salt, and the @ku.th check still works. But SRS-26 is not met until the password column is gone, and we are not claiming otherwise."

**At the midterm you said you checked roles on every request. But the code only had login checks.** Pakhin

"You are right, and we said so at the time. Before, we checked that you were logged in. We did not check what your role allowed. This iteration the permission check is real. In the demo you saw the same account treated differently after we promoted it, with nothing changing except one row in the database."

**Your page hides Edit Role from students. Is that your security?** Thanawin

"No. That is only what the user sees. The server refuses the change anyway, because anyone can call the endpoint directly. We hide the link so a student is not offered something that will only fail. If you want, I can call that endpoint from a student account and show you the refusal."

## Your report says testing is "none applicable". Kantanut

**If you wrote the notes:** "That line is out of date and we have corrected it. Our test notes are in the docs folder, and I can show them now."

**If you did not:** "That line is wrong, and it is my fault. We did check each story by hand as we built it —

**FILL: name two** — but we did not write the results down, so we have no evidence to show you. Writing that document is my first task next iteration."

## What is still not built? Kantanut

"**FILL: the current list**. US-16 is deferred by decision, not forgotten. And this iteration we **FILL: removed the**  
**Reset Code button, or built it**. We would rather remove a button than show one that does nothing."

## Before the day

### Fix the report — in this order

1. **Replace the repository link.** It currently points at a music website.
2. **Rewrite both critical paths** using your own tasks, not the restaurant example.
3. **Rewrite the US-5 and US-6 descriptions.** They still say "menu item", and the actor should be the lecturer.
4. **Rewrite the retrospective.** It is about a menu view you do not have.
5. **Replace "None Applicable" in testing.** One hour of writing is worth a whole band.
6. Fill in sections 6 and 7 — Risk Register and Contribution Log.
7. Fix the spelling of Kantanut's name on the cover.
8. Decide whether SRS-9 changes, now that the assignment fields are title, description, due date and assigned TA.

### Then check you all say the same thing

- Does every sentence in the deck match what the code really does?
- Do the deck, the report, the Plane board and the commits describe the same iteration?
- Does each person's contribution log point at commits from their own account?
- Are the requirement IDs you say the same as in the current SRS?

### Set up the demo

- Three accounts signed in: the owner, a student you will promote, a student who stays a student
- A classroom with at least one assignment already in it
- The app already running. Do not start Docker in front of the room
- Backup video ready, showing the same thing you plan to demo

### Practise

1. Once out loud, standing up, with a timer.
2. Time each segment on its own. The demo is what runs over.
3. Do the demo once with the internet off, to check the backup works.
4. Ask each other the seven questions above, cold, in random order.
5. Agree the handover sentence between each part, so there is no silence.

#### ONE IDEA HOLDS THE WHOLE PRESENTATION TOGETHER

Everything here says the same thing a different way: **we checked something in one place, when it needed to be guaranteed in another.** Hiding a button, instead of refusing on the server. A double-click fix in the browser, instead of a rule in the database. Knowing who you are, instead of knowing what you may do.

If a question surprises you, answer from that idea. It is true about your project, and it shows you understood this iteration, not just finished it.